

JavaScript II

External libraries and REST APIs

Web Programming I

UEF// University of Eastern Finland

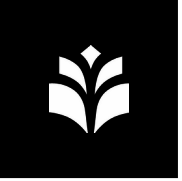
Course materials originally by Sonsoles López-Pernas, adapted by Maria Kuismin



JavaScript II

External libraries and REST APIs

Student is able to
integrate external APIs into a web application using asynchronous JavaScript.

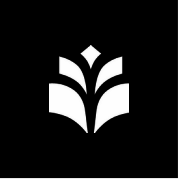


Content

JS Libraries
- Why use third-party JS libraries?
- What kinds of tasks can we delegate to libraries?

Asynchronous JavaScript
- What does asynchronous mean, and why does it matter?
- How do we deal with asynchronous code?

REST APIs
- What is a REST API, and why is it useful?
- How does a REST API work?
- How do we use a REST API in JavaScript?



JS Libraries

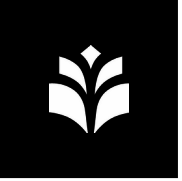
- Why use third-party JS libraries?
- What kinds of tasks can we delegate to libraries?

Asynchronous JavaScript

- What does asynchronous mean, and why does it matter?
- How do we deal with asynchronous code?

REST APIs

- What is a REST API, and why is it useful?
- How does a REST API work?
- How do we use a REST API in JavaScript?

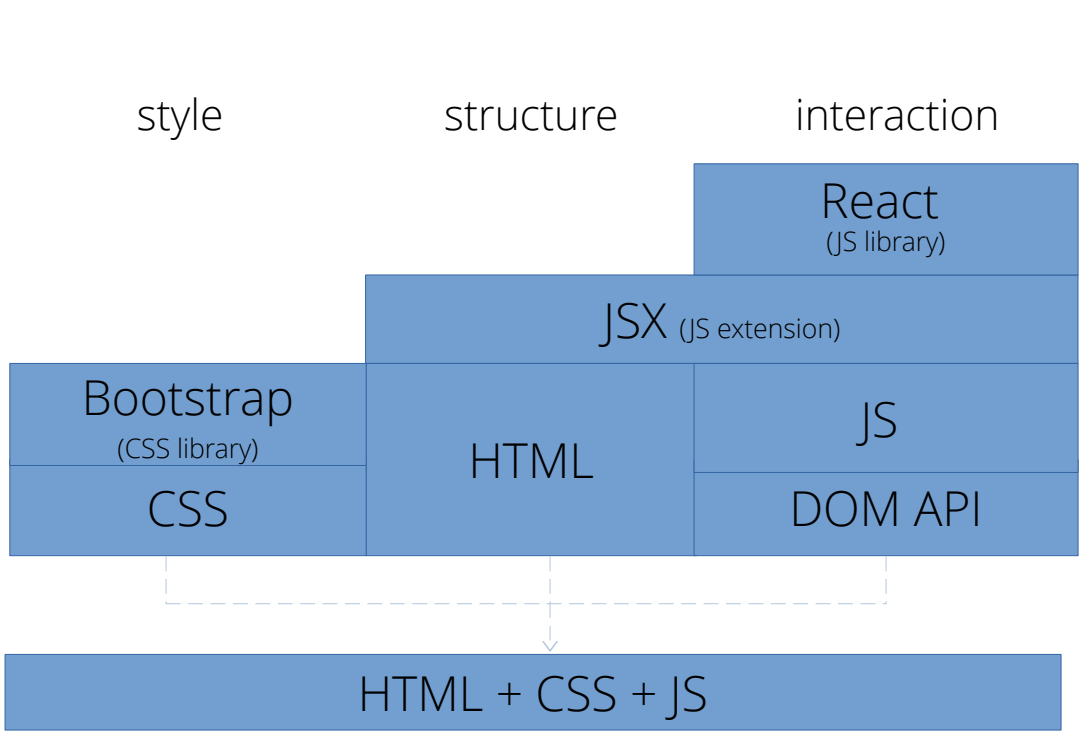


Benefits of using third-party libraries

Why not reinvent the wheel?

- faster development
- well-documented solutions
- battle-tested code
- bug fixes and maintenance
- browser quirks handled
- large user base

Review: Course technologies



collaboration
version control

GitHub

git

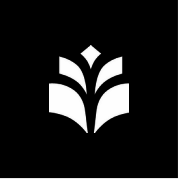
Libraries exist at different layers

CSS layer (styling and animation) → Bootstrap
e.g. Bootstrap, Tailwind CSS, Animate.css

JS + DOM layer (interaction and data) → focus of today's lecture

Component framework layer (application structure)
e.g. React, Angular, Vue.js → React in the next lectures

Development tools
e.g. Vite, npm, ESLint, Babel → we'll use these when starting React projects



Common tasks that JS libraries handle

- DOM manipulation
- Form validation
- Animations and transitions
- Advanced interactions
- Data visualization
- UI component interactions

Tasks: DOM manipulation

- DOM manipulation
- Form validation
- Animations and transitions
- Advanced interactions
- Data visualization
- UI component interactions

Selecting, changing, creating, and removing HTML elements.

Vanilla JS: document.getElementById(), querySelector(), textContent, createElement, appendChild, remove() → used for small projects and scripts

Component frameworks (React, Angular, Vue): Handle DOM updates through Virtual DOM. Developer describes what the UI should look like, framework figures out what to change
→ standard approach for modern web applications

Historically: jQuery simplified DOM manipulation
→ Still found in older projects and some libraries

All three approaches manipulate the DOM — the difference is how directly you do it.

Tasks: Form validation

- DOM manipulation
- Form validation
- Animations and transitions
- Advanced interactions
- Data visualization
- UI component interactions

Checking user input before sending it to the server.

HTML layer: built-in validation attributes required, pattern, min, max, type="email" → in HTML lecture

JS layer: custom validation logic with addEventListener → in JavaScript lecture

Libraries: Yup (schema validation), Validator.js; Formik, React Hook Form → used in React projects

Surveys: Survey.js — complete survey/form builder

Tasks: Animations and transitions

- DOM manipulation
- Form validation
- Animations and transitions
- Advanced interactions
- Data visualization
- UI component interactions

Making UI elements move, fade, or transform smoothly.

CSS layer: CSS transitions, keyframes, Animate.css
→ already possible with what learned in CSS lecture

JS layer: GSAP, Anime.js — for complex, programmatic animations that CSS alone cannot handle (e.g., physics-based motion, scroll-triggered animations)

* example on these slides

Tasks: Advanced interactions

- DOM manipulation
- Form validation
- Animations and transitions
- Advanced interactions
- Data visualization
- UI component interactions

Specialized UI features that would take too long to build from scratch.

Date/time pickers: Flatpickr

Toast (temporal) notifications Toastify

Image editing: Cropper.js

Rich text editors: Quill.js

Drag and drop: Sortable.js

3D graphics: Three.js

* example on these slides

Tasks: Data Visualization

- DOM manipulation
- Form validation
- Animations and transitions
- Advanced interactions
- **Data visualization**
- UI component interactions

Displaying data as charts, graphs, and diagrams.

JS layer: Chart.js — bar, line, pie, doughnut charts

React layer: Recharts — same idea but as React components → in React lectures

Network diagrams: Cytoscape.js

* example on these slides

Tasks: UI component interactions

- DOM manipulation
- Form validation
- Animations and transitions
- Advanced interactions
- Data visualization
- UI component interactions

Managing complex user interfaces with many interactive parts.

Examples: navigation menus, accordions, tabs, modals, shopping carts, form wizards

Simple cases: handled with vanilla JS (addEventListener,classList.toggle)

Complex cases: component frameworks like Angular, Vue, React → covered in React lectures

Library: GSAP.js

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script
src="https://cdn.jsdelivr.net/npm/gsap@3.12/dist/gsap.min.js"></script>
    <style>
        .like-btn.liked { color: red; border-color: red; }
    </style>
</head>

<body>

- - - -
```

Library: GSAP.js

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
    <script
src="https://cdn.jsdelivr.net/npm/gsap@3.12/dist/gsap.min.js"></script>
    <style>
        .like-btn.liked { color: red; border-color: red; }
    </style>
</head>

<body>
```

JS file from CDN provides the function

Toggles between `class="like-btn"` and `class="like-btn liked"`

```
</body>
</html>
```



Find the elements and add event listener

```
<body>
    <button class="like-btn" id="likeBtn">♡</button>
    <p id="likeCount">0 likes</p>

    <script>
        const btn = document.getElementById('likeBtn');
        const countEl =
document.getElementById('likeCount');
        let likes = 0;
        let liked = false;

        btn.addEventListener('click', () => {
            liked = !liked;
            likes += liked ? 1 : -1;
            btn.textContent = liked ? '♥' : '♡';
            btn.classList.toggle('liked');
            countEl.textContent = likes + ' likes';

            // GSAP animation: heart bounces on click
            gsap.fromTo(btn,
                { scale: 1 },
                { scale: 1.4, duration: 0.15, yoyo:
true, repeat: 1 }
            );
        });
    </script>
</body>
```

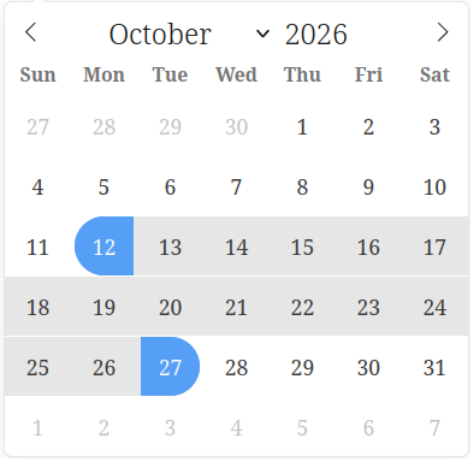
Toggle boolean

Animates the button — grows and shrinks back

Library: Flatpickr

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/flatpickr/dist/flatpickr.min.css">
    <script src="https://cdn.jsdelivr.net/npm/flatpickr"></script>
</head>
<body>
---------------------------------------->
</body>
</html>
```

Choose date range: 2026-10-12

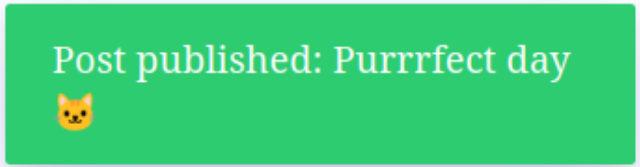



```
<body>
    <h1>Filter posts by date</h1>
    <label for="dateRange">Choose date range:</label>
    <input type="text" id="dateRange"
placeholder="Select date range">
    <script>
        flatpickr("#dateRange", {
            mode: "range",
            dateFormat: "Y-m-d"
        });
    </script>
</body>
```

Library: Flatpickr

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/flatpickr/dist/flatpickr.min.css">
    <script src="https://cdn.jsdelivr.net/npm/flatpickr"></script>
</head>
<body>
- - - -
```

Library: Toastr




```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <link rel="stylesheet" type="text/css"
href="https://cdn.jsdelivr.net/npm/toastify-js/src/toastify.min.c
ss">
    <script type="text/javascript"
src="https://cdn.jsdelivr.net/npm/toastify-js"></script>
</head>
<body>
  ---------------------------------------->
</body>
</html>
```

```
<body>
    <form id="postForm">
        <textarea id="caption" name="caption" rows="3"
required></textarea>
        <br><br>
        <button type="submit">Post</button>
    </form>

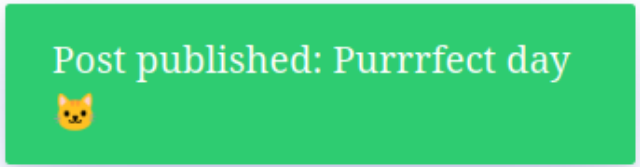
    <script>
        document.getElementById('postForm')
            .addEventListener('submit', function(e) {
                e.preventDefault();


                const caption =
document.getElementById('caption').value;

                Toastify({
                    text: "Post published: " + caption
                     + " 🐱",
                    duration: 3000,
                    style: { background: "#2ecc71" }
                }).showToast();

                this.reset();
            });
    </script>
</body>
```

Library: Toastr




```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <link rel="stylesheet" type="text/css"
href="https://cdn.jsdelivr.net/npm/toastify-js/src/toastify.min.c
ss">
    <script type="text/javascript"
src="https://cdn.jsdelivr.net/npm/toastify-js"></script>
</head>
<body>
</body>
</html>
```

CSS file from CDN provides the style and JS file from CDN provides the function

Find the element and add event listener

```
<body>
    <form id="postForm">
        <textarea id="caption" name="caption" rows="3"
required></textarea>
        <br><br>
        <button type="submit">Post</button>
    </form>

    <script>
        document.getElementById('postForm')
            .addEventListener('submit', function(e) {
                e.preventDefault();
```

Prevents page reload on form submit

```
                const caption =
document.getElementById('caption').value;

                Toastify({
                    text: "Post published: " + caption
                     + " 🐱",
                    duration: 3000,
                    style: { background: "#2ecc71" }
                }).showToast();

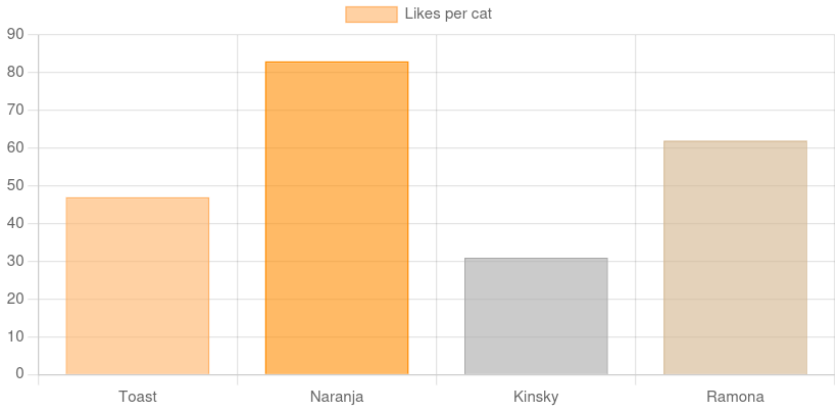
                this.reset();
            });
    </script>
</body>
```

Library function: creates the "toast"

and displays it on screen

Library: Charts.js

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Basic Chart.js Example</title>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
---------------------------------------->
</body>
</html>
```



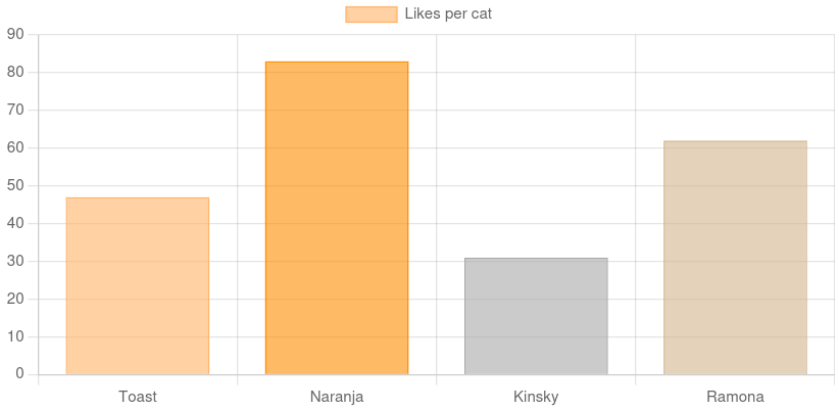

```
<body>
    <canvas id="myChart" width="400" height="200"></canvas>
    <script>
        const ctx =
document.getElementById('myChart').getContext('2d');
        const myChart = new Chart(ctx, {
        type: 'bar',
        data: {
            labels: ['Toast', 'Naranja', 'Kinsky', 'Ramona'],
            datasets: [{
            label: 'Likes per cat',
            data: [47, 83, 31, 62],
            backgroundColor: [
                'rgba(255, 178, 102, 0.6)',
                'rgba(255, 140, 0, 0.6)',
                'rgba(169, 169, 169, 0.6)',
                'rgba(210, 180, 140, 0.6)'
            ],
            borderColor: [
                'rgba(255, 178, 102, 1)',
                'rgba(255, 140, 0, 1)',
                'rgba(169, 169, 169, 1)',
                'rgba(210, 180, 140, 1)'
            ],
            borderWidth: 1
            }]
        }
        });
    </script>
</body>
```

Library: Charts.js

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
```

JS file from CDN provides the function

```
</body>
</html>
```



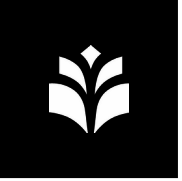
Select canvas and get drawing context

```
<body>
    <canvas id="myChart" width="400" height="200"></canvas>
    <script>
        const ctx =
document.getElementById('myChart').getContext('2d');
        const myChart = new Chart(ctx, {
        type: 'bar',
        data: {
            labels: ['Toast', 'Naranja', 'Kinsky', 'Ramona'],
            datasets: [{
            label: 'Likes per cat',
            data: [47, 83, 31, 62],
            backgroundColor: [
                'rgba(255, 178, 102, 0.6)',
                'rgba(255, 140, 0, 0.6)',
                'rgba(169, 169, 169, 0.6)',
                'rgba(210, 180, 140, 0.6)'
            ],
            borderColor: [
                'rgba(255, 178, 102, 1)',
                'rgba(255, 140, 0, 1)',
                'rgba(169, 169, 169, 1)',
                'rgba(210, 180, 140, 1)'
            ],
            borderWidth: 1
            }]
        }
        });
    </script>
</body>
```

creates the chart

Chart type (bar, line, pie, ...)

Labels and values for X- and Y-axis



Libraries exist for almost everything

Some more libraries worth knowing about:

Content creation

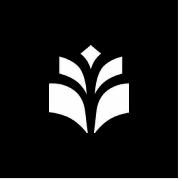
Rich text editors: Quill.js
Image cropping: Cropper.js
Surveys and forms: Survey.js

Interaction

Drag and drop: Sortable.js

Graphics and games

3D graphics: Three.js
Network diagrams: Cytoscape.js
2D game engine: Phaser



JS Libraries

- Why use third-party JS libraries?
- What kinds of tasks can we delegate to libraries?

Asynchronous JavaScript

- What does asynchronous mean, and why does it matter?
- How do we deal with asynchronous code?

REST APIs

- What is a REST API, and why is it useful?
- How does a REST API work?
- How do we use a REST API in JavaScript?

Enabling server communication without freezing the page

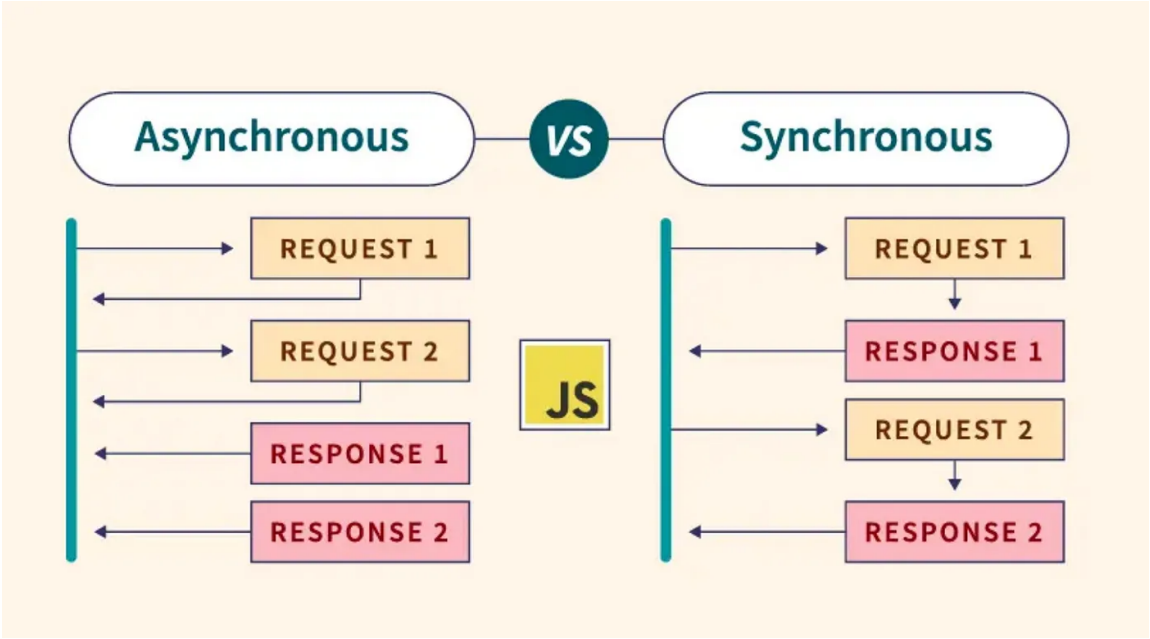
`addEventListener`, `textContent`, `classList.toggle` run instantly — no waiting needed
New situation: what if something takes time? Fetching data from a server can take 1–5 sec.

Without async:
- JavaScript waits for data
- Page freezes
- User cannot interact

With async:
- JavaScript starts the fetch and continues running other code
- Returns to data when ready
- Page stays responsive

The main uses:
→ Sending a form (payment, posting a photo, ...)
→ Sending messages
→ Using 3rd party REST APIs
→ Any communication with a server

Asynchronous requests



Code continues — like ordering food and doing other things while waiting

Code stops and waits — like standing in a queue doing nothing

Note: JavaScript is single-threaded — unlike Java threads, it does not run code in parallel. Instead, an event loop manages tasks cleverly so the page stays responsive.

https://gurindernarang.medium.com/asynchronous-programming-callbacks-async-await-promises-in-js-19e294e84a79

Three ways of dealing with asynchrony

These are not just three alternatives — they are a progression.
Each one solved problems of the previous.

1. Callbacks → first approach, works but gets messy
2. Promises → cleaner, but still complex to chain
3. async/await → modern standard, reads like regular code

We will look at each briefly, then focus on async/await.

Callbacks

A function passed as an argument to another function.
Called when the asynchronous task is done.

You already know one:
addEventListener is a callback

```
btn.addEventListener('click',
() => { ... });
```

↑ This function runs when the click happens — not immediately

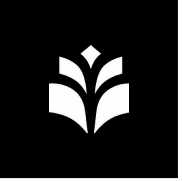
Another example: setTimeout

```
setTimeout(function() {
    alert("Time up!");
  }, 3000);
```

↑ This function runs after 3 seconds — not immediately

Problem with callbacks for data fetching:

```
getData(function(data) {
    processData(data, function(result) {
        saveResult(result,
function(response) {
            notifyUser(response, function()
{
// deeply nested callbacks – hard to read and debug
            });
        });
    });
});
```



Callbacks - deeply nested callbacks

```
getData(function(data) {
    if (!data) return handleError('getData failed');  // error handling at every
level
    processData(data, function(result) {
        if (!result) return handleError('processData failed');  // error
handling at every level
        saveResult(result, function(response) {
            if (!response) return handleError('saveResult failed');  // error
handling at every level
            notifyUser(response, function() {
                // deeply nested callbacks — hard to read and debug
                // error handling must be repeated at every level
            });
        });
    });
});
```

Promises

A Promise represents the eventual result of an async operation.
States: pending → fulfilled or rejected

Better than deeply nested callbacks

```
getData()
    .then(data => processData(data))     // when data arrives
    .then(result => saveResult(result))
    .then(response => notifyUser(response))
    .catch(error => handleError(error)) // one catch for the whole chain
```

One .catch() handles errors from anywhere in the chain – no need to repeat error handling at every level

But: if different steps need each other's data, variables need to be shared across .then() step → async/await solves this

Promises - sharing data between steps

```
// if different steps need each other's data
getData()
    .then(data => {
        storedData = data;  // must be stored in an outer variable
        return processData(data);
    })
    .then(result => {
        // both result and the original storedData are needed here
        return saveResult(result, storedData);
    })
    .then(response => notifyUser(response))
    .catch(error => handleError(error));
// Outer variable needed because .then() steps
// cannot directly access each other's data
// → async/await keeps all variables in the same scope
```

Async/await – the modern standard

Built on top of Promises — reads like regular sequential code.

Rules:

→ define the function with async

→ use await before the slow operation

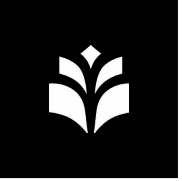
→ wrap in try/catch for error handling (same as Java)

→ the rest of the page continues normally while waiting

```
async function getCats() {
  try {
    const response = await
fetch('https://api.example.com/cats');
    const cats = await response.json();
    console.log(cats);  // use the data here
  } catch (error) {
    console.error('Something went wrong:',
error);
  }
}
getCats();  // call the function
```

Async/await – all variables in same scope

```
async function doEverything() {
    try {
        const data = await getData();
        const result = await processData(data);
        // data is still accessible here — same scope
        const response = await saveResult(result, data);
        await notifyUser(response);
    } catch (error) {
        // one catch handles all errors
        handleError(error);
    }
}
doEverything();
// Compare:
// Callbacks → deeply nested, error handling repeated at every level
// Promises  → flat chain, but data sharing requires outer variables
// Async/await → flat, all variables in same scope, one try/catch
```



JS Libraries

- Why use third-party JS libraries?
- What kinds of tasks can we delegate to libraries?

Asynchronous JavaScript

- What does asynchronous mean, and why does it matter?
- How do we deal with asynchronous code?

REST APIs

- What is a REST API, and why is it useful?
- How does a REST API work?
- How do we use a REST API in JavaScript?

The problem – Need of real data

What we know so far:
JavaScript can manipulate the DOM — changing what is on the page.
But where does the data come from?

Our JavaScript so far has used hardcoded data.

Real applications need real, up-to-date data:

→ movie information
→ weather forecasts
→ maps and locations
→ payment processing
→ anything that changes over time or belongs to a user

The solution — REST APIs

A REST API is an address (URL) that another server provides.
(Representational State Transfer Application Programming Interface)

How it works:
1. Our JavaScript sends an HTTP request to that URL
2. The server processes the request
3. The server responds with data in JSON format
4. We use the data to update the DOM

Connection to what you already know:
- HTTP requests: GET, POST, PUT, DELETE
- JSON format: { "name": "Toast", "age": 6, "likes": 47 }
- Fetching data uses async/await

→ because waiting for a server response takes time

Examples of REST APIs

OpenStreetMap / Nominatim — maps, addresses, directions openstreetmap.org / nominatim.openstreetmap.org

REST Countries — country information restcountries.com

OpenSky Network — real-time flight data opensky-network.org

Open Library — book information, covers, author data openlibrary.org/developers

Wikipedia API — article summaries and content en.wikipedia.org/api/rest_v1/

OMDB API — movie information

Resource and HTTP method

Data is treated as a resource — identified by a unique URL.
HTTP methods define what you want to do with that resource.

GET → retrieve data "give me the list of books"
POST → create new data "add this new book"
PUT → update data "change this book's information"
DELETE → remove data "remove this book"

/books example endpoint of an API for a library

GET	/books	Lists all the books in the database
GET	/books/{bookId}	Retrieves a book based on their id
DELETE	/books/{bookId}	Deletes a book based on their id
POST	/books	Creates a Book
PUT	/books/{bookId}	Method to update a book

→ pattern:
URL identifies the resource,
HTTP method defines the action

Data format — JavaScript Object Notation

REST APIs typically respond with data in JSON format (JavaScript Object Notation)

```
{
  "name": "Toast",
  "age": 6
}
```

Same format as JavaScript objects: `let cat = { name: "Toast", age: 6 };`

JSON vs. XML — the same data, different format:

XML (older, heavier):

```
<cat>
  <name>Toast</name>
  <age>46</age>
</cat>
```

JSON (modern, lightweight):

```
{
  "name": "Toast",
       "age": 6
}
```

JSON is preferred because it is lighter and directly compatible with JavaScript objects.

Server remembers nothing between requests

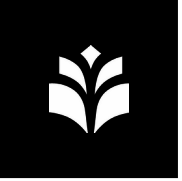
Statelessness

Each request from a client to the server must be independent and contain all the necessary information for the server to fulfill that request.

The server does not remember previous requests or store session information.

in practice

→ every request you send must include authentication (if needed)
→ the server treats each request independently
→ this is why APIs often require an API key in every request



Using a REST API in JavaScript: fetch()

fetch() is a modern built-in JavaScript function for HTTP requests.

```
async function getData() {
  try {
    const response = await fetch('https://api.example.com/data'); // URL of the REST API
    // GET by default — no method specified
    if (!response.ok) {
      throw new Error(`HTTP error! Status: ${response.status}`); // detect possible errors
    }

    const data = await response.json(); // extract the JSON response (needs await too)

    console.log(data);
    // use the data — for example, update the DOM

  } catch (error) {
    console.error('Error fetching data:', error); // handle errors
  }
}

getData(); // call the function — e.g. after clicking a button
```

GET as default, others must be set explicitly

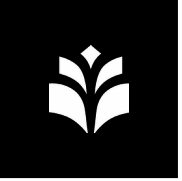
By default fetch uses GET — to retrieve data:

```
fetch('https://api.com/data');
```

For POST — to send data to the server:

```
await fetch('https://api.com/data', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    name: 'Toast',
    email: 'toast@purr.pur'
  })
});
```

`method: 'POST'` → tells the server this is a create request
`headers` → tells the server the data format is JSON
`body: JSON.stringify()` → converts JS object to JSON string



Both syntaxes work for async functions

```
async function getData() {
  const res = await fetch("https://my-api-url.com/");
  const data = await res.json();
  setProducts(data);
}
```

```
const getData = async () => {
  const res = await fetch("https://my-api-url.com/");
  const data = await res.json();
  setProducts(data);
}
```

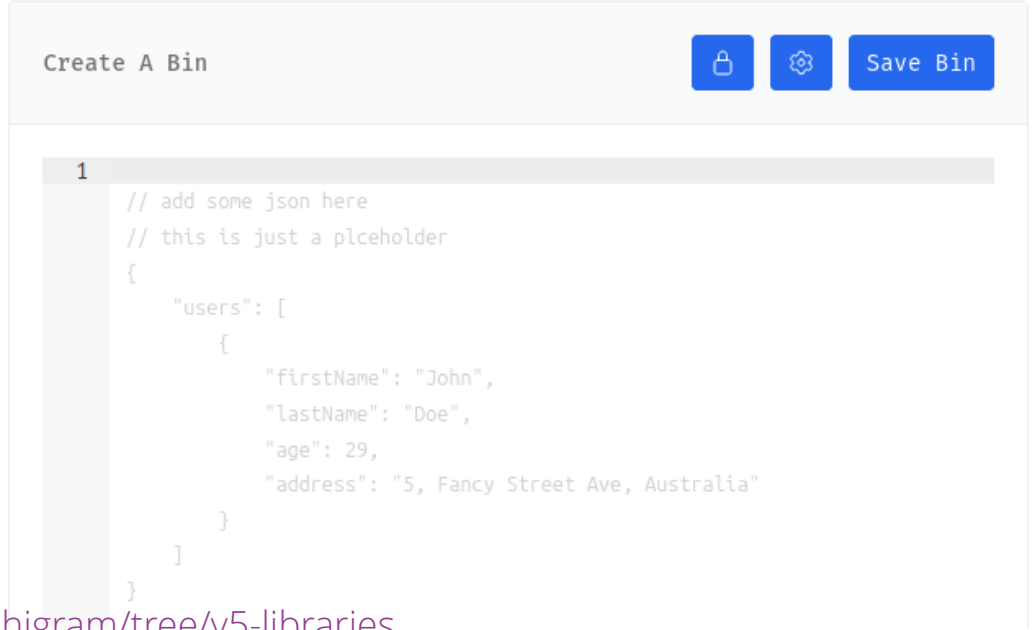
Summary

- Libraries handle common tasks e.g. data visualization, date pickers, UI component interactions — saving time and providing tested code.
- Asynchronous JavaScript lets long-running tasks run in the background without blocking other code; approaches developed from callbacks, to promises and finally async/await.
- REST APIs enable integration of external services (e.g., public APIs) and persistent data storage.
- REST APIs are a way for systems to communicate over HTTP. Data is a resource (URL), action is defined by HTTP method (GET, POST, PUT, DELETE), data exchanged in JSON
- fetch is the modern built-in JavaScript function for making HTTP requests to REST APIs, used with async/await.

Example: Using jsonbin.io with Kažigram

Jsonbin.io provides a simple REST API for storing JSON data.

1. Create an account at jsonbin.io
2. Create a bin, a storage location for JSON
3. Retrieve API key and bin ID



Each "bin" stores one JSON document.
GET → retrieve the entire bin (all posts at once)
PUT → replace the entire bin
(send the whole updated array)
Headers include the API key
= statelessness in practice – every request must authenticate itself

Starting point: https://github.com/kuisdemo/kazhigram/tree/v5-libraries